

ASSIGNMENT 2

Graph Theory and Route Optimization

1. INTRODUCTION

This project models a smart transportation network using Graph Theory concepts. A synthetic city network consisting of 500 cities is generated and represented as a weighted undirected graph. Each city is represented as a vertex, while roads between cities are represented as weighted edges containing distances.

The objective of this project is to analyze network connectivity, detect clusters of connected cities, and determine the shortest routes between cities using classical graph algorithms.

2. DATA STRUCTURE USED

Adjacency List Representation

The transportation network is stored using an adjacency list. Each city maintains a list of its neighboring cities along with the distance to each neighbor.

Example:

City 0 -> (City 1, 15 km), (City 3, 25 km)

Advantages:

- Memory efficient for sparse graphs
- Faster traversal of neighboring vertices
- Suitable for large networks

Space Complexity:

$O(V + E)$

where:

V = Number of vertices (cities)

E = Number of edges (roads)

3. TASK A: NETWORK GENERATION

Algorithm Used:

Random Graph Generation

Description:

A network of 500 cities is created. Each city is randomly connected to between 2 and 6 other cities. Every road is assigned a random distance between 5 km and 50 km.

Steps:

1. Create 500 city nodes.
2. Generate random neighboring cities.
3. Assign random edge weights.
4. Store all connections in an adjacency list.

Time Complexity:

$O(V \times K)$

where K is the average number of connections per city.

Since K is small and bounded (2–6):

Time Complexity $\approx O(V)$

Space Complexity:

$O(V + E)$

4. TASK B: CONNECTIVITY ANALYSIS

Algorithm Used:

Depth First Search (DFS)

Purpose:

DFS is used to identify all cities that belong to the same connected region of the network.

Working:

1. Start from an unvisited city.
2. Mark it as visited.
3. Visit all connected neighboring cities recursively.

4. Continue until no unvisited neighbors remain.

The largest connected component is identified by comparing the size of each discovered component.

Time Complexity:

$O(V + E)$

Reason:

Each vertex and edge is visited at most once.

Space Complexity:

$O(V)$

Reason:

Visited array and recursion stack.

5. TASK C: CLUSTER DETECTION

Algorithm Used:

Connected Component Detection using DFS

Purpose:

To identify separate groups of cities that are connected internally but disconnected from other groups.

Outputs:

- Total number of connected components
- Size of each component
- Largest cluster
- Smallest cluster

Working:

1. Traverse every city.
2. If a city is unvisited, start DFS.
3. Store all cities discovered by DFS into one component.
4. Repeat until all cities are visited.

Time Complexity:

$O(V + E)$

Space Complexity:

$O(V)$

6. TASK D: SHORTEST PATH ANALYSIS

Algorithm Used:

Dijkstra's Algorithm

Purpose:

To determine the shortest distance from a selected source city to every other reachable city.

Working:

1. Assign distance 0 to source city.
2. Assign infinity to all other cities.
3. Use a priority queue to select the city with minimum distance.
4. Relax neighboring edges.
5. Continue until all shortest distances are computed.

Applications:

- GPS Navigation
- Logistics Route Planning
- Delivery Optimization
- Emergency Response Systems

Time Complexity:

$O(E \log V)$

Reason:

Priority Queue operations require logarithmic time.

Space Complexity:

$O(V)$

Reason:

Distance array and priority queue storage.

7. ALGORITHMIC PERFORMANCE

Algorithm Time Complexity Space Complexity

DFS $O(V + E)$ $O(V)$

Connected Components
(using DFS) $O(V + E)$ $O(V)$

Dijkstra $O(E \log V)$ $O(V)$

Adjacency List $O(V + E)$ $O(V + E)$

8. REAL-WORLD APPLICATIONS

1. Smart Transportation Systems
 - Route optimization
 - Traffic management
2. Logistics and Delivery
 - Efficient package delivery
 - Fuel cost reduction
3. Emergency Services
 - Fastest path for ambulances
 - Disaster response planning
4. Digital Mapping Services
 - Navigation systems
 - Location-based services

9. CONCLUSION

This project demonstrates the practical application of Graph Theory in transportation and logistics systems. By using adjacency lists, Depth First Search, Connected Component Analysis, and Dijkstra's Algorithm, the transportation network can be efficiently analyzed for connectivity, clustering, and route optimization. The implemented algorithms provide scalable solutions suitable for large real-world transportation networks.